

Este documento traduce los componentes técnicos del API Gateway a capacidades funcionales comprensibles para cualquier miembro del equipo. Cada funcionalidad se asocia con su módulo técnico y el beneficio concreto que aporta.

Funcionalidad	Módulo Técnico	Archivos Clave
Beneficio		
Proxy Inverso Transparente	<code>src/proxy/engine.ts</code> , <code>src/proxy/pool.ts</code>	ProxyEngine, Connection-PoolManager
Redirige peticiones HTTP de clientes a múltiples backends sin que el cliente conozca la topología interna. Centraliza un único punto de acceso para todos los microservicios.	Enrutamiento Dinámico por Prefijo	<code>src/routing/registry.ts</code> , <code>src/routing/matcher.ts</code>
RouteRegistry	Permite definir múltiples rutas (<code>\public</code> , <code>\secure</code> , <code>\api</code>) en un archivo YAML. Cada prefijo apunta a un backend diferente con configuración independiente.	Autenticación JWT por Ruta
<code>src/middleware/jwt-auth/plugin.ts</code>	JwtAuthPlugin	Protege rutas individuales exigiendo un token Bearer válido. Inyecta los claims decodificados como headers seguros al backend, eliminando la necesidad de que cada microservicio implemente su propia lógica de autenticación.
Rate Limiting Distribuido	<code>src/middleware/rate-limit/plugin.ts</code> <code>src/middleware/rate-limit/store.ts</code>	RateLimitPlugin, RedisRateLimitStore
Limita el número de peticiones por IP en una ventana de tiempo configurable. Funciona de forma distribuida gracias a Redis, garantizando coherencia en despliegues con múltiples replicas.	Circuit Breaker con Reinicios	<code>src/middleware/circuit-breaker/plugin.ts</code> , <code>src/middleware/circuit-breaker/state.ts</code> , <code>src/middleware/circuit-breaker/retry.ts</code>

Funcionalidad	Modulo Tecnico	Archivos Clave
Beneficio		
CircuitBreakerPlugin, CircuitStateMachine	Protege al Gateway y a los backends de fallos en cascada. Si un backend deja de responder, el circuito se abre y devuelve errores locales rapidos (503), evitando consumir recursos en conexiones inutilles. Incluye reintentos automaticos con backoff exponencial.	Metricas Prometheus Nativas
src/middleware/metrics/plugin.ts	MetricsPlugin	Expone un endpoint <code>\metrics</code> compatible con Prometheus que incluye contadores de peticiones, histogramas de latencia y tasas de error, desglosados por ruta, metodo HTTP y backend.
Recarga de Configuracion en Caliente	src/config/reloader.ts	ConfigReloader
Permite modificar parametros operativos (limites de tasa, nivel de log) sin reiniciar el proceso, enviando la senal <code>SIGHUP</code> . Ideal para ajustes en produccion sin tiempo de inactividad.	Logging Estructurado JSON	src/logger/setup.ts
createLogger (Pino)	Emite logs en formato JSON optimizado para ingesta por herramientas de observabilidad (ELK, Datadog, Cloud-Watch). El nivel de log es dinamicamente reconfigurable.	Validacion Estricta de Configuracion
src/config/schema.ts , src/config/loader.ts	GatewayConfigSchema (Zod)	Garantiza que toda la configuracion sea valida antes de que el Gateway acepte trafico. Un campo malformado detiene el arranque inmediatamente, evitando estados inconsistentes en produccion.
Graceful Shutdown	src/index.ts	gracefulShutdown()

Funcionalidad	Modulo Tecnico	Archivos Clave
Beneficio		
Asegura un apagado limpio del proceso: drena peticiones activas, cierra pools de conexiones y libera la conexion a Redis antes de terminar. Evita la perdida de datos y conexiones huérfanas durante despliegues.	Connection Pooling Inteligente	<code>src/proxy/pool.ts</code>
ConnectionPoolManager	Mantiene pools de sockets TCP/TLS persistentes por cada backend, reutilizando conexiones entre peticiones. Reduce drasticamente la latencia de red y el consumo de descriptors de archivo del sistema operativo.	

El siguiente diagrama muestra el camino que recorre una petición desde que el cliente la envía hasta que recibe la respuesta del backend:



El sistema de middlewares está diseñado como un pipeline de plugins donde cada módulo implementa la interfaz `GatewayPlugin`:

```

interface GatewayPlugin {
  name: string;
  onRequest?(context: RequestContext): Promise<void>;
  onResponse?(context: ResponseContext): Promise<void>;
}

```

Para agregar una nueva funcionalidad al Gateway (por ejemplo, un middleware de CORS, un transformador de respuestas o un cache en memoria), basta con:

1. Crear una clase que implemente `GatewayPlugin`.
2. Implementar `onRequest()` para la lógica pre-proxy o `onResponse()` para la lógica post-proxy.
3. Registrar la instancia en el array `pluginsList` en `src/index.ts`.

El pipeline ejecuta los plugins en el orden de registro. Si un plugin responde directamente al cliente (`reply.sent == true`), el pipeline se detiene y no ejecuta los plugins restantes (short-circuit).